

1. ¿Cuáles son los tipos de Datos en Python?

- **int:** Enteros (ej: 42, -10).
- **float:** Números decimales (ej: 3.14, 2.0).
- **str:** Cadenas de texto (ej: "Hola Mundo").
- **list:** Listas mutables (ej: [1, 2, "Python"]).
- **tuple:** Secuencias inmutables (ej: (10, 20)).
- **dict:** Diccionarios (pares clave-valor, ej: {"nombre": "Ana", "edad": 25}).
- **bool:** Verdadero o Falso (True, False).
- **set:** Conjuntos de elementos únicos (ej: {1, 2, 3}).

2. ¿Qué tipo de convención de nomenclatura deberíamos utilizar para las variables en Python?

En Python, la guía de estilo oficial es la PEP 8. Para variables, la convención es:

- **snake_case:** Debes usar minúsculas separadas por guiones bajos.
 - *Bien:* nombre_usuario, total_calculado.
 - *Mal:* camelCase o PascalCase.
- **Constantes:** Se escriben totalmente en mayúsculas.
 - *Ejemplo:* IVA_VALOR = 0.16.

3. ¿Qué es un Heredoc en Python?

Python no tiene el concepto exacto de "Heredoc" (como en Bash), pero logramos exactamente el mismo resultado utilizando cadenas de caracteres de triple comilla (''' o ''').

Se usan principalmente para documentos multilínea o cadenas de documentación (docstrings):

```
mensaje = """
Esta es una cadena
que abarca múltiples líneas
sin necesidad de caracteres de escape.
"""
```

4. ¿Qué es una interpolación de cadenas?

Es el proceso de insertar variables o expresiones directamente dentro de una cadena de texto. La forma más moderna y recomendada en Python (desde la versión 3.6) son las f-strings:

```
nombre = "Carlos"
edad = 30
# Interpolación con f-string
print(f"Hola, mi nombre es {nombre} y tengo {edad} años.")
```

5. ¿Cuándo deberíamos usar comentarios en Python?

Usa comentarios cuando:

- La lógica sea compleja o poco intuitiva.
- Estés aplicando un "parche" temporal (etiquetado como `# TODO:` o `# FIXME:`).
- Estés escribiendo Docstrings (comentarios de triple comilla al inicio de funciones o clases para documentar qué hacen).

6. ¿Cuáles son las diferencias entre aplicaciones monolíticas y de microservicios?

La arquitectura monolítica es la forma tradicional de construir aplicaciones, donde toda la lógica de negocio, el acceso a bases de datos y la interfaz de usuario se encuentran unidos en una única unidad de despliegue. Imagínalo como una gran caja donde todo está interconectado; si deseas realizar un cambio en una pequeña parte del código, necesitas reconstruir y desplegar toda la aplicación nuevamente. Aunque al inicio del desarrollo es más sencillo de gestionar, a medida que el proyecto crece, el "monolito" se vuelve difícil de mantener, y un error en un módulo puede comprometer la estabilidad de todo el sistema.

Por otro lado, los microservicios proponen una estructura modular. Aquí, la aplicación se divide en múltiples servicios pequeños, independientes y especializados, que se comunican entre sí, generalmente a través de APIs o colas de mensajes. Cada servicio se encarga de una función específica y puede ser desarrollado, probado y desplegado de forma autónoma. Esto permite que el equipo de desarrollo escale solo la parte de la aplicación que recibe más carga, lo cual es mucho más eficiente. Sin embargo, esta flexibilidad tiene un costo: la complejidad operativa aumenta significativamente, ya que ahora debes gestionar la comunicación y consistencia entre muchos servicios distintos.

